

UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE MATEMÁTICA E ESTATÍSTICA
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

Rafael Vieira de Carvalho

Divisão de nós em uma max-tree

Atualização dinâmica de max-trees sob transformações anti-extensivas locais

Orientador: Prof. Dr. Ronaldo F. Hashimoto
Coorientador: Prof. Dr. Wonder A. L. Alves

**São Paulo
2026**

1 Resumo

2 Introdução e Justificativas

- ~~Motivação para o que queremos estudar~~
 - Explicar o que é morfologia matemática
 - Histórico de EDs utilizadas no passado para morfologia matemática
 - Explicar sobre component trees
 - Explicar hipóteses necessárias para construção de component tree
 - Explicar para que component tree é utilizada
 - Explicar por que é bastante utilizada [34, 26, 10, 27]
- ~~O que é o problema da reconstrução~~
- Qual é o estado da arte "merge"
- Qual a lacuna que pretende-se pesquisar "split"
- explicar como está dividido esse documento

A morfologia matemática é uma subárea do processamento de imagens e da visão computacional que tem como objetivo estudar e desenvolver transformações aplicadas a imagens, tendo como principal arcabouço teórico a teoria dos conjuntos [8]. Conforme sugere o próprio termo *morfologia*, essa área busca analisar e extrair informações relacionadas às características geométricas dos objetos presentes nas imagens, como forma, tamanho e estrutura.

Nessa busca pelo desenvolvimento de transformações, o uso de estrutura de dados, para representar e aplicar transformações em imagens, é extremamente necessário. Exemplos de estruturas são: quad-trees [15, 3], binary partition tree [11], watershed tree [7]

A *component tree* [12] é uma das estruturas utilizadas para representar imagens. A ideia fundamental por trás dessa estrutura consiste em representar uma imagem a partir da ordenação de seu conjunto de pixels e de suas componentes conexas. Para isso, é necessário definir o conceito de componente conexa em uma imagem [10, 9] e, de forma mais geral, estabelecer como uma imagem pode ser representada por meio de um grafo [8].

A representação de uma imagem como um grafo é realizada de forma direta, na qual os vértices correspondem aos pixels da imagem, enquanto as arestas são definidas de acordo com um determinado modelo de conectividade [10].

Conforme dito anteriormente, uma das premissas para a construção da *component tree* é a ordenação do conjunto de pixels. Existem duas formas de ordenar os pixels: de maneira ascendente ou de maneira descendente. Essas duas formas dão origem a duas estruturas de dados duais, a *max tree* cuja ordenação é ascendente, ou seja, a relação de ordenação dos pixels se dá pelo operador $\leq$ entre os níveis de cinza. E a *min tree* cuja ordenação é descendente, ou seja, a relação de ordenações dos pixels se dá pelo operador $\geq$ entre os níveis de cinza.

A *component tree* originalmente foi desenhada para realizar operações conexas com o propósito de resolver tarefas de filtragem e segmentação [13, 14]. Para realizar tais tarefas a operação mais comum utilizada é o corte, que pode consistir na remoção de alguns nós baseado na otimização de algum critério ou na remoção de alguns nós baseado nos atributos contidos nesses nós.

Tal estrutura é amplamente utilizada devido ao seu baixo custo computacional de construção. Segundo [6], uma imagem com n pixels pode ter sua *component tree* construída em tempo $O(n \log n)$. Além dessa característica, a *component tree* permite extrair informações morfológicas relevantes das imagens. Por esse motivo, essa estrutura tem sido empregada em abordagens de aprendizado profundo para a extração de características, conforme apresentado em [1, 2, 5].

Nesse contexto, uma dificuldade encontrada na utilização de *component trees* é a atualização dessa estrutura quando a imagem sofre alguma alteração. Isto é, quando a imagem sofre alguma alteração não existe nenhum algoritmo para atualizar *component tree* de modo a representar a imagem transformada. Dessa forma, nesse caso de uso se faz necessário reconstruir a árvore novamente para cada mudança que ocorre na imagem.

Em [4] os autores propõem a operação de *merge* (mescla) de nós para resolver parcialmente o problema de atualização. Como apontado também em [4] o problema dual, *split*/divisão, também precisa ser resolvido para poder-se atualizar completamente a árvore.

Sob esse contexto, esse projeto de pesquisa pretende contribuir com essa lacuna na literatura. Para simplificar, assim como [4] trata apenas de *max tree*, esse projeto tratará também apenas desse tipo de estrutura.

Este documento está organizado da seguinte forma. Na Seção 3, será apresentada a pergunta de pesquisa, bem como as perguntas auxiliares que a orientam. Para isso, serão introduzidas algumas das notações, propriedades e definições apresentadas em [4]. Na Seção 4, serão descritos os métodos que serão empregados para alcançar os objetivos propostos, bem como os procedimentos adotados para garantir a reprodutibilidade da pesquisa. Na Seção 5, será apresentado o planejamento das atividades e a priorização das etapas a serem executadas, de modo a assegurar o desenvolvimento adequado da pesquisa. Por fim, na Seção 6, serão discutidos os possíveis riscos associados à pesquisa e propostas estratégias de mitigação e alternativas para os casos em que o planejamento originalmente estabelecido não produza os resultados esperados.

3 Objetivos

- Construir arcabouço teórico
- **Objetivo Geral:** Declarar de forma direta o que pretende-se entregar
- **Objetivo Específico:** Mapear os passos concretos necessários para alcançar o objetivo geral (fundamentação matemática, algoritmo, suporte a atributos, demonstrações de corretude/complexidade).

4 Metodologia

Tópicos para a construção de metodologia

- Construção do arcabouço teórico para elaborar o algoritmo
- Demonstrações do arcabouço teórico
- Elaboração do algoritmo
- Demonstração de corretude do algoritmo
- Análise da complexidade computacional do algoritmo
- Implementação do algoritmo
- Elaboração de métricas para o teste do algoritmo

- Teste do algoritmo em algum dataset (a ser definido)
- Teste utilizando o rebuild
- Comparação entre rebuild e split
- estudo de caso com split e merge (no exemplo do paper de rebuild utiliza o filtro em sequência)

5 Plano de trabalho e cronograma

6 Mitigação de Riscos

Referências

- [1] Wonder A.L. Alves, Charles F. Gobber, Dennis J. Silva, Alexandre Morimitsu, Ronaldo F. Hashimoto, and Beatriz Marcotegui. Image segmentation based on ultimate levelings: From attribute filters to machine learning strategies. *Pattern Recognition Letters*, 133:264–271, May 2020.
- [2] D. L. Farfan Cabrera, N. Gogin, D. Morland, B. Naegel, D. Papathanassiou, and N. Passat. Segmentation of Axillary and Supraclavicular Tumoral Lymph Nodes in PET/CT: A Hybrid CNN/Component-Tree Approach. In *2020 25th International Conference on Pattern Recognition (ICPR)*, pages 6672–6679, Milan, Italy, January 2021. IEEE.
- [3] Gregory M. Hunter and Kenneth Steiglitz. Operations on Images Using Quad Trees. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2):145–153, April 1979.
- [4] Wonder Alexandre Luz Alves, Nicolas Passat, Dennis J. Silva, Alexandre Morimitsu, and Ronaldo F. Hashimoto. Component Tree: Update Rather than Rebuild. *Journal of Mathematical Imaging and Vision*, 68(5):61, August 2026.
- [5] Cyril Meyer, Étienne Baudrier, Patrick Schultz, and Benoît Naegel. Combining Max-Tree and CNN for Segmentation of Cellular FIB-SEM Images. In Bertrand Kerautret, Miguel Colom, Adrien Krähenbühl, Daniel Lopresti, Pascal Monasse, and Benjamin Perret, editors,

Reproducible Research in Pattern Recognition, volume 14068, pages 77–90. Springer Nature Switzerland, Cham, 2023.

- [6] L. Najman and M. Couprie. Building the Component Tree in Quasi-Linear Time. *IEEE Transactions on Image Processing*, 15(11):3531–3539, November 2006.
- [7] L. Najman and M. Schmitt. Geodesic saliency of watershed contours and hierarchical segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 18(12):1163–1173, December 1996.
- [8] Laurent Najman and Jean Cousty. A graph-based mathematical morphology reader. *Pattern Recognition Letters*, 47:3–17, October 2014.
- [9] Benjamin Perret. Inf-structuring Functions: A Unifying Theory of Connections and Connected Operators. *Journal of Mathematical Imaging and Vision*, 51(1):171–194, January 2015.
- [10] Azriel Rosenfeld. Connectivity in Digital Pictures. *Journal of the ACM*, 17(1):146–160, January 1970.
- [11] P. Salembier and L. Garrido. Binary partition tree as an efficient representation for image processing, segmentation, and information retrieval. *IEEE Transactions on Image Processing*, 9(4):561–576, April 2000.
- [12] P. Salembier, A. Oliveras, and L. Garrido. Antiextensive connected operators for image and sequence processing. *IEEE Transactions on Image Processing*, 7(4):555–570, April 1998.
- [13] P. Salembier and J. Serra. Flat zones filtering, connected operators, and filters by reconstruction. *IEEE Transactions on Image Processing*, 4(8):1153–1160, August 1995.
- [14] Philippe Salembier and Michael Wilkinson. Connected operators. *IEEE Signal Processing Magazine*, 26(6):136–157, November 2009.
- [15] S. Tanimoto and T. Pavlidis. A hierarchical data structure for picture processing. *Computer Graphics and Image Processing*, 4(2):104–119, June 1975.